



UNIVERSIDAD DE LAS FUERZAS ARMADAS

SEDE SANTO DOMINGO

GITHUB:

https://github.com/Yeikolon/proyecto_clase_api_paso_a_paso.git

ASIGNATURA:

Fundamentos de sistemas web

ESTUDIANTE:

Yeiko Velez

NRC:

29535

DOCENTE:

Ing. Geovanny Brito

FECHA:

25/07/2026

SANTO DOMINGO - ECUADOR

Índice de Contenido

Resolución documentada de ejercicios de JavaScript.....	4
Introducción	4
Desarrollo paso a paso	4
Mini proyecto 1: Ejecutar código automáticamente al cargar la página	4
Mini proyecto 2: Modificar el HTML con JavaScript a partir de un botón	5
Mini proyecto 3: Leer datos desde un objeto tipo JSON embebido en JavaScript	6
Mini proyecto 4: Recorrer un JSON anidado y calcular un promedio	7
Mini proyecto 5: Consumir una API externa con fetch.....	8
Mini proyecto 6: Cargar una imagen automáticamente desde una API	9
Mini proyecto 7: Cargar un archivo JSON local con fetch	10
Mini proyecto 8: Proyecto integrador simple: JSON local + API externa.....	1
Mini proyecto 9: Catálogo de eventos básicos del DOM.....	2
Mini proyecto 10: Eventos + DOM + JSON embebido: tabla dinámica de estudiantes.....	3
Mini proyecto 11: Eventos combinados con peticiones asíncronas a distintas APIs	5
Mini proyecto 12: Catálogo de productos desde la API de DummyJSON	6
Mini proyecto 13: Proyecto integrador final: sesión, APIs, librerías externas y gráfico	7
Código relevante explicado	9
Capturas de pantalla	9
Conclusiones	20

Índice de figuras

Figura 1. DOM.....	10
Figura 2. Alerta	10
Figura 3. Leer json	10
Figura 4. Calcular promedio de json.....	11
Figura 5. Consumir API.....	11
Figura 6. Cargar Imagen	11
Figura 7. Cargar json local.....	12
Figura 8. Ejercicios combinados.....	13
Figura 9. Eventos varios A.....	14
Figura 10. Eventos varios B.....	14
Figura 11. Eventos + DOM + json.....	15
Figura 12. APIS	15

Figura 13. Eventos	16
Figura 14. Generar catalogo a partir de una API	16
Figura 15. Inicio de sesion	17
Figura 16. Países con su bandera	18
Figura 17. Clima	18
Figura 18. Mensaje de confirmación	19
Figura 19. Grafica	19

Índice de códigos

Código 1. Ejercicio 1	5
Código 2. Ejercicio 2	6
Código 3. Ejercicio 3	7
Código 4. Ejercicio 4	8
Código 5. Ejercicio 5	9
Código 6. Ejercicio 6	10
Código 7. Ejercicio 7	1
Código 8. Ejercicio 8	2
Código 9. Ejercicio 9	3
Código 10. Ejercicio 10	5
Código 11. Ejercicio 11	6
Código 12. Ejercicio 12	7
Código 13. Ejercicio 13	9

Resolución documentada de ejercicios de JavaScript

Introducción

Este informe documenta el proceso de aprendizaje seguido a lo largo de 13 mini proyectos desarrollados con HTML, CSS y JavaScript. Cada mini proyecto se construyó sobre el anterior, avanzando de forma progresiva desde conceptos básicos —como ejecutar código al cargar la página o responder a un clic— hasta prácticas más completas que combinan eventos del DOM, estructuras JSON, consumo de APIs externas mediante fetch con async/await, almacenamiento en el navegador (sessionStorage y localStorage) y librerías externas como Bootstrap, SweetAlert2, Toastify y Chart.js.

El objetivo general de esta serie de prácticas es comprender cómo JavaScript transforma una página HTML estática en una aplicación web dinámica, capaz de reaccionar a las acciones del usuario, procesar datos estructurados y comunicarse con servicios externos.

Desarrollo paso a paso

Mini proyecto 1: Ejecutar código automáticamente al cargar la página

Objetivo

Comprender que un script no se ejecuta línea por línea de forma inmediata en toda circunstancia, sino que puede esperar a que el documento HTML termine de cargar.

Qué se desarrolló

Se creó una página con un contenedor vacío (<div id="resultado">) y un archivo app.js que escribe un texto dentro de ese contenedor apenas el navegador termina de construir el DOM.

Lógica JavaScript aplicada

Se usa el evento **DOMContentLoaded** del objeto document para registrar una función que se ejecutará automáticamente cuando el HTML esté listo, sin esperar a imágenes u otros recursos externos. Dentro de esa función se localiza el elemento por su id y se reemplaza su contenido de texto con textContent.

Eventos utilizados

- DOMContentLoaded (se dispara una sola vez, al terminar de cargar el HTML)

Funciones / conceptos clave

- document.addEventListener()
- document.getElementById()
- elemento.textContent

Fragmento de código más importante

Código 1. Ejercicio 1

```
document.addEventListener("DOMContentLoaded", iniciarAplicacion);

function iniciarAplicacion(){
  const resultado = document.getElementById("resultado"); resultado.textContent =
  "Elking";
}
```

Mini proyecto 2: Modificar el HTML con JavaScript a partir de un botón

Objetivo

Aprender a capturar un evento de interacción del usuario (un clic) y usarlo para modificar el contenido de la página en tiempo real.

Qué se desarrolló

Se agregó un botón (btnMostrar) y un contenedor de resultado. Al presionar el botón, JavaScript cambia el texto del contenedor y muestra una alerta emergente.

Lógica JavaScript aplicada

Se obtiene la referencia al botón con getElementById y se le asocia un *listener*. Cuando el evento ocurre, la función accion() actualiza el DOM y dispara un alert() nativo del navegador.

Eventos utilizados

- click (evento de interacción directa del usuario con el botón)

Funciones / conceptos clave

- addEventListener()
- elemento.textContent
- alert()

Fragmento de código más importante

Código 2. Ejercicio 2

```
const btnMostrar = document.getElementById("btnMostrar");
btnMostrar.addEventListener("click", accion);

function accion() {
  const resultado = document.getElementById("resultado"); resultado.textContent =
  "YAAAAAAA";
  alert("Holaaaaaa");
}
```

Mini proyecto 3: Leer datos desde un objeto tipo JSON embebido en JavaScript

Objetivo

Practicar la lectura de propiedades de un objeto (estructura idéntica a un JSON) y mostrarlas en pantalla usando plantillas de texto.

Qué se desarrolló

Se definió un objeto estudiante con propiedades simples (nombre, carrera, semestre) y arreglos (materias, notas). Al presionar un botón, se muestra la información del estudiante dentro de una tarjeta.

Lógica JavaScript aplicada

Se usa **notación de punto** (estudiante.nombre) para acceder a las propiedades del objeto y **template literals**

(comillas invertidas) para construir HTML dinámico que luego se inyecta con innerHTML.

Eventos utilizados

- click sobre el botón "Mostrar estudiante"

Funciones / conceptos clave

- addEventListener()
- innerHTML
- template literals (`\${}`)

Fragmento de código más importante

Código 3. Ejercicio 3

```
const estudiante = { nombre: "Anahi",
apellido: "Correa Delgado", carrera: "ITIN",
materias: ["programacion", "base de datos", "redes"], notas: [19, 18, 16, 17]
};
boton.addEventListener("click", mostrar); function mostrar() {
resultado.innerHTML = `
<p><strong>Nombre:</strong> ${estudiante.nombre} ${estudiante.apellido}</p>
`;
}
```

Mini proyecto 4: Recorrer un JSON anidado y calcular un promedio

Objetivo

Acceder a información ubicada varios niveles dentro de un objeto anidado y aplicar un cálculo sobre un arreglo.

Qué se desarrolló

Se construyó un objeto con una estructura más compleja, similar a la que entregaría una API real. Al presionar el botón, se ubica al estudiante "Luis Mora" y se calcula el promedio de sus notas.

Lógica JavaScript aplicada

Se combinan índices de arreglo y notación de punto (datos.institucion.estudiantes[1].materias[0].notas) para llegar hasta el dato buscado. Después se recorre el arreglo de notas con un ciclo **for...of**, acumulando la suma en una variable, y se divide entre la cantidad de notas para obtener el promedio.

Eventos utilizados

- click sobre el botón "Calcular promedio de Luis"

Funciones / conceptos clave

- addEventListener()
- for...of
- arreglo.length
- textContent

Fragmento de código más importante

Código 4. Ejercicio 4

```
function calcular(){ let suma = 0;
let datosEstudiante = datos.institucion.estudiantes[1].materias[0].notas;

for(const dato of datosEstudiante){ suma = dato + suma;
}
let promedio = suma / datosEstudiante.length; resultado.textContent = `Promedio:
${promedio}`;
}
```

Mini proyecto 5: Consumir una API externa con fetch

Objetivo

Dar el primer paso hacia el consumo de servicios web reales: pedir datos a un servidor externo y esperar la respuesta antes de continuar.

Qué se desarrolló

Se consume la API pública catfact.ninja/fact, que devuelve una curiosidad aleatoria sobre gatos en formato JSON, y se muestra en pantalla al presionar un botón.

Lógica JavaScript aplicada

La función consultar() se declara **async** para poder usar **await**. Primero se espera la respuesta de fetch() (un objeto Response) y luego se espera la conversión de esa respuesta a JSON con .json(). Mientras la petición viaja, se muestra el mensaje "Cargando..." para dar retroalimentación visual al usuario.

Eventos utilizados

- click sobre el botón "Cargar dato desde API"

Funciones / conceptos clave

- async / await
- fetch()
- respuesta.json()
- console.log()

Fragmento de código más importante

Código 5. Ejercicio 5

```
async function consultar(){ resultado.innerHTML = "Cargando...";  
  
const respuesta = await fetch("https://catfact.ninja/fact"); const datos = await  
respuesta.json();  
  
resultado.textContent = datos.fact;  
}
```

Mini proyecto 6: Cargar una imagen automáticamente desde una API

Objetivo

Reutilizar la misma función tanto para la carga automática al iniciar la página como para la recarga manual mediante un botón.

Qué se desarrolló

Se consume la API `dog.ceo/api/breeds/image/random`, que retorna la URL de una imagen aleatoria de un perro. La imagen se carga sola al abrir la página y también puede recargarse con un botón.

Lógica JavaScript aplicada

La misma función `generarImagen()` se registra en **dos eventos distintos**: `DOMContentLoaded` (para la carga inicial) y `click` del botón "Cargar otra imagen" (para repetir la acción). Esto evita duplicar código.

Eventos utilizados

- `DOMContentLoaded`
- `click` sobre el botón "Cargar otra imagen"

Funciones / conceptos clave

- `async / await`
- `fetch()`
- `imagen.src`
- reutilización de funciones

Fragmento de código más importante

Código 6. Ejercicio 6

```
async function generarImagen() {  
  estado.textContent = "Cargando imagen Perro desde la API...";  
  const respuesta = await fetch("https://dog.ceo/api/breeds/image/random");  
  const datos = await respuesta.json();  
  
  imagen.src = datos.message;  
  estado.textContent = "Imagen cargada correctamente";  
}
```

Mini proyecto 7: Cargar un archivo JSON local con fetch

Objetivo

Aprender que fetch también puede leer archivos del propio proyecto (no solo servidores externos), lo cual es la base para separar datos y lógica.

Qué se desarrolló

Se creó el archivo data/academico.json con información de una institución y sus estudiantes. Al hacer doble clic en un botón, la aplicación lee ese archivo y muestra un resumen.

Lógica JavaScript aplicada

Se usa `fetch("./data/academico.json")` para leer el archivo de forma asíncrona. Como los navegadores bloquean la lectura de archivos locales abiertos con doble clic (protocolo `file://`), este proyecto requiere ejecutarse con un servidor local (Live Server) para que la petición funcione sin errores en consola.

Eventos utilizados

- `dblclick` sobre el botón "Cargar JSON local"

Funciones / conceptos clave

- `fetch()` con ruta relativa
- `async/await`
- `innerHTML`
- `template literals`

Fragmento de código más importante

Código 7. Ejercicio 7

```
boton.addEventListener("dblclick", cargar);

async function cargar() {
  const respuesta = await fetch("./data/academico.json"); const datos = await
  respuesta.json();

  const institucion = datos.institucion.nombre; const estudiantes =
  datos.institucion.estudiantes;

  resultado.innerHTML = `<h2>${institucion}</h2>
  <p>Estudiantes: ${estudiantes.length}</p>
  <p>Primer estudiante: ${estudiantes[0].nombre}</p>`;
}
```

Mini proyecto 8: Proyecto integrador simple: JSON local + API externa

Objetivo

Integrar en una sola página los tres aprendizajes previos: cargar una imagen desde una API, leer un JSON local y calcular un promedio, todo coordinado desde una única función de arranque.

Qué se desarrolló

La página se organiza en tres secciones (tarjetas): imagen automática, promedio de un estudiante y una curiosidad desde API. Una función iniciarAplicacion() coordina la carga inicial y registra los tres botones.

Lógica JavaScript aplicada

Se declara una variable global datosAcademicos que guarda el JSON una vez descargado, para no tener que volver a pedirlo cada vez que se calcula el promedio. Se valida con un if que los datos ya estén disponibles antes de calcular, evitando así un error si el usuario presiona el botón demasiado rápido.

Eventos utilizados

- DOMContentLoaded
- click (tres botones independientes)

Funciones / conceptos clave

- async/await
- fetch() (local y externo)
- variables globales para caché de datos
- for...of
- validación con if

Fragmento de código

Código 8. Ejercicio 8

```
function iniciarAplicacion(){ generarImagen(); cargarJSONAcademico();
btnImagen.addEventListener("click", generarImagen);
btnPromedio.addEventListener("click", calcularPromedio);
btnDato.addEventListener("click", generarDato);
}

function calcularPromedio(){ if(datosAcademicos === null){
alert("No hay datos académicos"); return;
}
const notas = datosAcademicos.institucion.estudiantes[1].materias[0].notas;
let suma = 0;
for(const nota of notas){ suma += nota; }
resultadoPromedio.textContent = "El promedio de Luis Mora es " + (suma /
notas.length);
}
```

Mini proyecto 9: Catálogo de eventos básicos del DOM

Objetivo

Practicar, en una sola página, los eventos más usados en formularios e interfaces: click, doble clic, mouseover/mouseout, input, change, keydown y submit.

Qué se desarrolló

Se construyó una página con Bootstrap que reúne seis mini-demostraciones independientes: dos botones (click y doble clic), una tarjeta que reacciona al pasar el mouse, un campo de texto que refleja lo escrito, un selector de carrera, un campo que detecta la tecla Enter y un formulario que se envía sin recargar la página.

Lógica JavaScript aplicada

Cada demostración se organiza en su propia función (configurarEventosClick, configurarEventosMouse, etc.) que se invoca desde el evento DOMContentLoaded. Esto mantiene el código ordenado y cada evento tiene una responsabilidad clara. En el formulario se usa event.preventDefault() para evitar que la página se recargue al enviarlo.

Eventos utilizados

- click
- dblclick
- mouseover / mouseout
- input
- change
- keydown
- submit

Funciones / conceptos clave

- addEventListener()
- classList.add()/remove()
- event.key

Fragmento de código más importante

Código 9. Ejercicio 9

```
tarjetaMouse.addEventListener("mouseover", function(){ tarjetaMouse.textContent
= "El mouse esta sobre la caja"; tarjetaMouse.classList.add("activa");
});

formularioDemo.addEventListener("submit", function(event){
event.preventDefault(); resultadoFormulario.textContent =
"Formulario capturado con JavaScript. Correo: " + correo.value;
});
```

Mini proyecto 10: Eventos + DOM + JSON embebido: tabla dinámica de estudiantes

Objetivo

Construir una mini aplicación de gestión de datos: mostrar, buscar, filtrar, calcular promedios y

agregar registros a una tabla, todo a partir de un arreglo de objetos en memoria.

Qué se desarrolló

Se definió un arreglo estudiantes (nombre, carrera y notas) que se representa en una tabla HTML. Tres botones permiten mostrar todos los registros, calcular promedios o limpiar la tabla; un campo de búsqueda filtra por nombre en tiempo real; un selector filtra por carrera; y un formulario permite agregar un nuevo estudiante.

Lógica JavaScript aplicada

La función pintarTabla(lista) recibe cualquier arreglo (completo o filtrado) y reconstruye las filas de la tabla con forEach y plantillas de texto. Los filtros usan Array.prototype.filter() para generar un nuevo arreglo según el texto escrito o la carrera elegida, sin modificar el arreglo original. El formulario valida que los campos no estén vacíos antes de usar push() para añadir el nuevo estudiante y vuelve a pintar la tabla.

Eventos utilizados

- click (3 botones)
- input (búsqueda en vivo)
- change (filtro por carrera)
- submit (agregar estudiante)

Funciones / conceptos clave

- Array.filter()
- Array.forEach()
- Array.push()
- event.preventDefault()
- template literals
- toFixed()

Fragmento de código más importante

Código 10. Ejercicio 10

```
buscador.addEventListener("input", function () { const texto =
buscador.value.toLowerCase();
const filtrados = estudiantes.filter(function (est) { return
est.nombre.toLowerCase().includes(texto);
});
pintarTabla(filtrados);
});

formEstudiante.addEventListener("submit", function (event) {
event.preventDefault();
estudiantes.push({ nombre, carrera, notas: [nota] }); pintarTabla(estudiantes);
});
```

Mini proyecto 11: Eventos combinados con peticiones asíncronas a distintas APIs***Objetivo***

Practicar cómo distintos tipos de evento (click, change, submit, keydown) pueden disparar peticiones fetch hacia diferentes servicios externos.

Qué se desarrolló

La página reúne cinco demostraciones: un botón que trae una curiosidad de gato, un botón que trae una imagen de perro, un selector que decide qué API consultar, un formulario que busca un usuario por id en una API de usuarios de prueba, y un campo de texto que informa la última tecla presionada.

Lógica JavaScript aplicada

Cada evento dispara su propia función async. En el buscador de usuario se valida primero que el id esté entre 1 y 10 antes de consultar `jsonplaceholder.typicode.com/users/{id}`; una vez llega la respuesta, se muestran nombre, correo y ciudad, y se revela la tarjeta oculta quitándole la clase `none` de Bootstrap.

Eventos utilizados

- click (2 botones)
- change (selector de API)
- submit (buscar usuario)
- keydown (registro de última tecla)

Funciones / conceptos clave

- async/await
- fetch()
- classList.remove()
- event.preventDefault()
- validación de rango numérico

Fragmento de código más importante

Código 11. Ejercicio 11

```

buscador.addEventListener("input", function () { const texto =
buscador.value.toLowerCase();
const filtrados = estudiantes.filter(function (est) { return
est.nombre.toLowerCase().includes(texto);
});
pintarTabla(filtrados);
});

formEstudiante.addEventListener("submit", function (event) {
event.preventDefault();
estudiantes.push({ nombre, carrera, notas: [nota] }); pintarTabla(estudiantes);
});

```

Mini proyecto 12: Catálogo de productos desde la API de DummyJSON

Objetivo

Consumir una API que entrega un arreglo de varios elementos y generar dinámicamente una tarjeta visual por cada uno, con manejo de errores.

Qué se desarrolló

Al presionar "Cargar productos", la aplicación consulta dummyjson.com/products, recorre el arreglo products de la respuesta y crea una tarjeta Bootstrap por cada producto con imagen, categoría, título, descripción, precio y stock.

Lógica JavaScript aplicada

Se usa try/catch alrededor de la petición para capturar fallos de red y se revisa respuesta.ok para detectar respuestas HTTP no exitosas antes de seguir procesando. Por cada producto se llama a crearTarjetaProducto(), que construye un elemento con document.createElement, arma su HTML interno con plantillas de texto y lo agrega al contenedor con appendChild.

Eventos utilizados

- click sobre el botón "Cargar productos"

Funciones / conceptos clave

- async/await
- fetch()
- try/catch
- respuesta.ok
- for...of
- document.createElement()
- appendChild()

Fragmento de código más importante

Código 12. Ejercicio 12

```
async function cargarProductos() { try{

const respuesta = await fetch("https://dummyjson.com/products");
if(!respuesta.ok){ throw new Error("No se obtuvo la información"); }

const datos = await respuesta.json(); for(const producto of datos.products){
crearTarjetaProducto(producto);
}
}catch(error){
estadoCarga.textContent = "No se pudo cargar los productos";
}
}
```

Mini proyecto 13: Proyecto integrador final: sesión, APIs, librerías externas y gráfico

Objetivo

Cerrar el curso integrando en una mini aplicación de dos pantallas todo lo aprendido: almacenamiento en el navegador, consumo de varias APIs, librerías externas y visualización de datos.

Qué se desarrolló

index.html simula un inicio de sesión: valida campos, guarda el nombre del usuario en sessionStorage y en localStorage, y redirige a home.html. Allí, un panel con pestañas permite:

consultar un listado de países y guardar la nacionalidad elegida, consultar el clima actual de una ciudad de Ecuador, ver un aviso de "Escribiendo..." con temporizador, mostrar confirmaciones y notificaciones con librerías externas, actualizar un gráfico de barras con datos aleatorios y visualizar el contenido guardado en ambos tipos de almacenamiento.

Lógica JavaScript aplicada

El estado de sesión se protege revisando `sessionStorage.getItem("usuarioActual")` al cargar `home.html`: si no existe, se redirige de vuelta al login (`window.location.href`). El efecto "Escribiendo..." se logra combinando el evento `input` con `setTimeout` y `clearTimeout`, reiniciando el temporizador cada vez que el usuario sigue escribiendo. Las notificaciones usan las librerías externas **SweetAlert2** (confirmaciones) y **Toastify** (avisos flotantes), y el gráfico se dibuja con **Chart.js**, actualizando su arreglo de datos con números aleatorios y llamando a `.update()`.

Eventos utilizados

- click (múltiples botones y pestañas)
- change (selector de país/ciudad)

Funciones / conceptos clave

- `sessionStorage` / `localStorage`
- `setTimeout` / `clearTimeout`
- `fetch()` a dos APIs
- librerías externas: Bootstrap, SweetAlert2, Toastify, Chart.js
- `window.location.href` (redirección)

Fragmento de código más importante

Código 13. Ejercicio 13

```

formLogin.addEventListener("submit", function(event) {
  event.preventDefault();
  if(nombreUsuario.value.trim() === "" || claveUsuario.value.trim() === ""){
    Toastify({ text: "Por favor llene los campos", duration: 3000
  }).showToast(); return;
  }
  sessionStorage.setItem("usuarioActual", nombreUsuario.value);
  localStorage.setItem("ultimoUsuario", nombreUsuario.value);
  window.location.href = "home.html";
});

campoMensaje.addEventListener("input", function() {
  estadoMensaje.textContent = "Escribiendo...";
  if (temporizador) clearTimeout(temporizador);
  temporizador = setTimeout(function() { estadoMensaje.textContent = ""; },
1000);
});

```

Código relevante explicado

A lo largo del curso se repiten cuatro patrones de código que sostienen la mayoría de los mini proyectos:

- **Selección de elementos + eventos:** `document.getElementById()` junto con `addEventListener()` es la base de toda interacción, desde el mini proyecto 1 hasta el 13.
- **Lectura de estructuras JSON:** ya sea un objeto embebido en el propio archivo JavaScript (pasos 3, 4 y 10) o un archivo `.json` leído con `fetch` (pasos 7 y 8), el acceso a los datos se hace combinando notación de punto, corchetes e índices de arreglo.
- **Peticiones asíncronas con `async/await`:** a partir del paso 5, casi todos los mini proyectos usan `fetch()` junto con `await` para esperar la respuesta de una API antes de actualizar la pantalla, siempre dentro de una función marcada como `async`.
- **Actualización dinámica del DOM:** ya sea reemplazando texto (`textContent`), HTML (`innerHTML`) o creando elementos nuevos (`createElement + appendChild`), todos los mini proyectos terminan mostrando su resultado modificando el árbol del documento.

Capturas de pantalla

Figura 1. DOM



Figura 2. Alerta

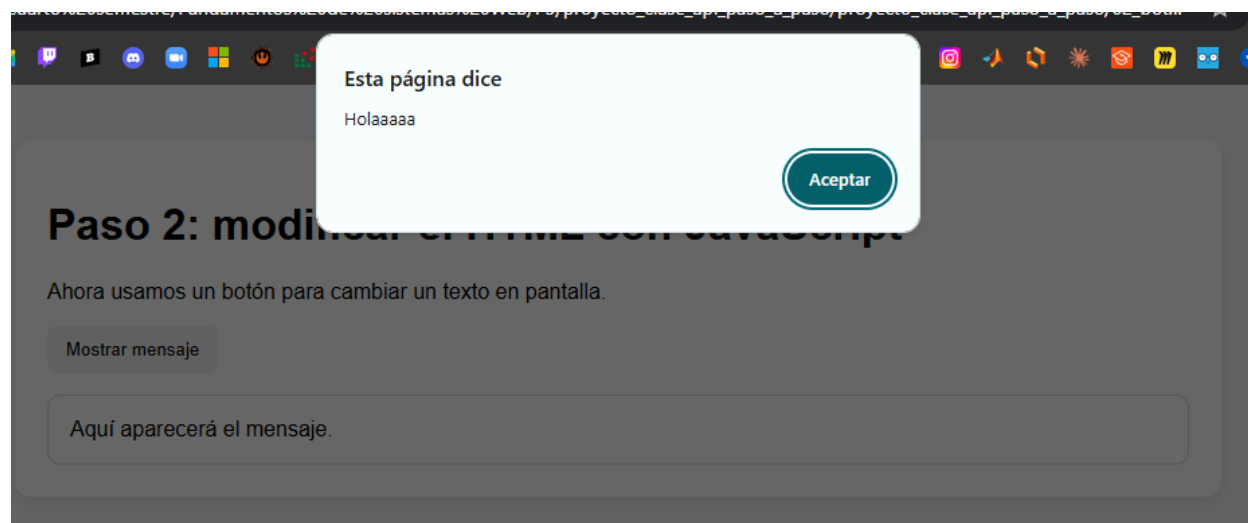


Figura 3. Leer json



Figura 4. Calcular promedio de json

Paso 4: calcular promedio desde un JSON anidado

Se accede al estudiante Luis Mora y se calcula su promedio.

Calcular promedio de Luis

Promedio: 14

Figura 5. Consumir API

Paso 5: traer datos desde una API

Se consume una API externa y se muestra una curiosidad de gatos.

Cargar dato desde API

Cats dislike citrus scent.

Figura 6. Cargar Imagen

Paso 6: cargar una imagen automáticamente

La imagen aparece sin presionar botón porque se carga al iniciar la página.

Imagen cargada correctamente



Cargar otra imagen

Figura 7. Cargar json local

Paso 7: cargar un archivo JSON local

Este ejemplo lee el archivo data/academico.json usando fetch.

Cargar JSON local

Universidad de práctica

Estudiantes: 2

Primer estudiante: Ana Torres

Figura 8. Ejercicios combinados

Paso 8: proyecto final simple

Integra JSON local, API externa e interacción con botones.

Imagen automática

Cargando imagen..1



Otra imagen

Promedio de Luis Mora

Calcular promedio

El promedio de Luis Mora es 14

Curiosidad desde API

Cargar dato

Approximately 24 cat skins can make a coat.

Figura 9. Eventos varios A

Parte 9: Eventos básicos en JavaScript

Esta parte sirve para probar eventos sencillos antes de trabajar con JSON o APIs.

La página ya está cargada

1. Evento click y doble click

Hacer click

Hacer doble click

El usuario dió 2 clicks

2. Eventos mouseover y mouseout

El mouse essta sobre la caja

Figura 10. Eventos varios B

3. Evento input

Escriba su nombre

asas

Texto escrito: **asas**

4. Evento change

Seleccione una carrera

Software

Carrera seleccionada: **Software**

5. Evento keydown

Escriba algo y presione Enter

b

Última tecla presionada: **Enter**

6. Evento submit

yevelez@espe.edu.ec

Enviar formulario

Formulario capturado con JavaScript. Correo: yevelez@espe.edu.ec

Figura 11. Eventos + DOM + json

Parte 10: Eventos + DOM + JSON embebido

En esta parte los eventos se usan para mostrar, buscar, filtrar y calcular datos.

Mostrar estudiantes

Calcular promedios

Limpiar tabla

Buscar por nombre

Filtrar por carrera

Todas

▼

Agregar estudiante con submit

Nombre

Carrera... ▼

Nota

Agregar

Mostrando promedios calculados.

#	Nombre	Carrera	Notas	Promedio
1	Ana Torres	Tecnologías de la Información	18, 17, 19	18.00
2	Luis Mora	Software	14, 13, 15	14.00
3	María Salazar	Sistemas	16, 15, 20	17.00

Figura 12. APIS

Parte 11: Eventos + API + async/await

En esta parte los eventos disparan peticiones externas usando fetch.

Aplicación lista. Interactúe con los controles.

1. Click: traer curiosidad de gato

Traer cat fact

Both humans and cats have identical regions in the brain responsible for emotion.

2. Click: traer imagen de perro

Cargar imagen de perro



Figura 13. Eventos

3. Change: seleccionar API a consultar

Cat Fact

The oldest cat on record was Crème Puff from Austin, Texas, who lived from 1967 to August 6, 2005, three days after her 38th birthday. A cat typically can live up to 20 years, which is equivalent to about 96 human years.

4. Submit y keydown

5

Buscar usuario

Chelsey Dietrich

Correo: Lucio_Hettinger@annie.ca

Ciudad: Roscoevew

5. Evento keydown

Escriba cualquier texto:

vb|

Última tecla presionada: Meta

Figura 14. Generar catalogo a partir de una API

Parte 12: Productos desde una API

En esta práctica se consume la API de DummyJSON y se recorren los productos para construir tarjetas visuales.

Catálogo de productos

Productos cargados 30

Cargar productos



beauty

Essence Mascara Lash Princess

Descripción: The Essence Mascara Lash Princess is a popular mascara known for its volumizing and lengthening effects. Achieve dramatic lashes with this long-lasting and cruelty-free formula.



beauty

Eyeshadow Palette with Mirror

Descripción: The Eyeshadow Palette with Mirror offers a versatile range of eyeshadow shades for creating stunning eye looks. With a built-in mirror, it's convenient for on-the-go makeup application.



beauty

Powder Canister

Descripción: The Powder Canister is a finely milled setting powder designed to set makeup and control shine. With a lightweight and translucent formula, it provides a smooth and matte finish.

Figura 15. Inicio de sesión

Práctica final

Integración final con JavaScript

Ingresa un nombre para simular una sesión de usuario.

Nombre del estudiante

Yeiko Velezc

Escribiendo...

Contraseña de práctica

.....

Iniciar práctica

Generar número aleatorio

Código generado: 2748

Figura 16. Países con su bandera

Carpetas 13

Integración final de JavaScript

Eventos, storage, APIs, librerías y gráfico.

Usuario: **Usuario: Yeiko Velezc**

Cerrar sesión simulada

Total consultas país

250

Último país

Bulgaria

Última ciudad clima

No disponible

Última temperatura

No disponible

Países

Clima

Temporizador

Alertas

Gráfico

Storage

API de países con bandera en el selector

El selector debe mostrar el país con su bandera al lado, por ejemplo: ec Ecuador.

Seleccione nacionalidad

BG Bulgaria

Guardar nacionalidad

Bandera: BG

País: Bulgaria

Capital: Sofia

Región: Europe

Figura 17. Clima

Carpetas 13

Integración final de JavaScript

Eventos, storage, APIs, librerías y gráfico.

Usuario: **Usuario: Yeiko Velezc**

Cerrar sesión simulada

Total consultas país

250

Último país

Bulgaria

Última ciudad clima

-1.03

Última temperatura

No disponible°C

Países

Clima

Temporizador

Alertas

Gráfico

Storage

API de clima actual

Seleccione una ciudad de Ecuador y consulte datos actuales de clima.

Ciudad

Quevedo

Consultar clima

Ciudad: -1.03

Temperatura: No disponible°C

Viento: No disponible km/h

Figura 18. Mensaje de confirmación

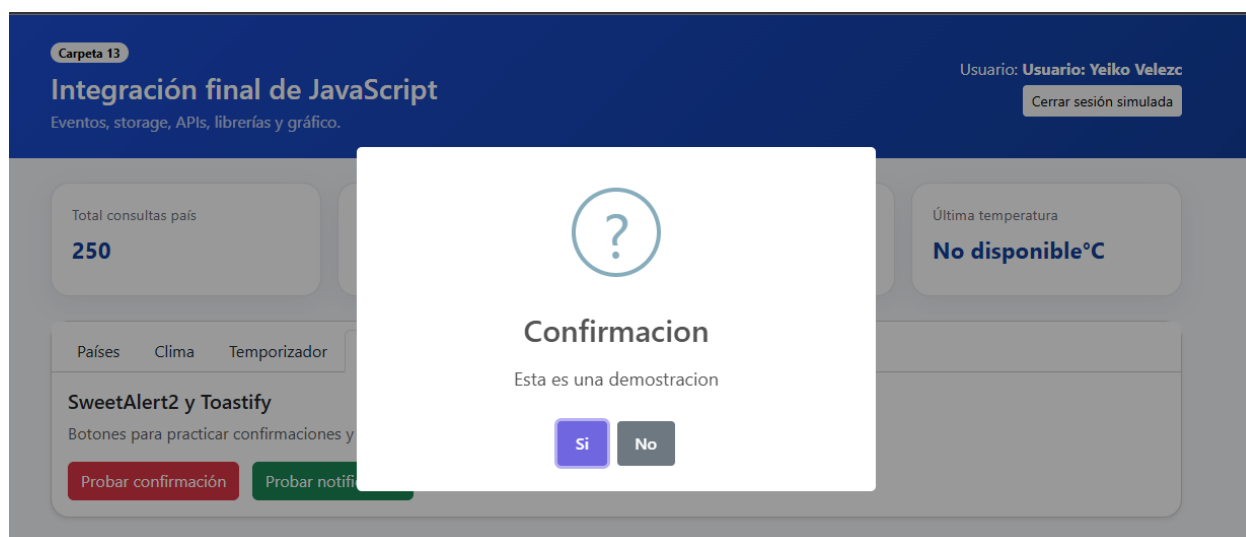


Figura 19. Grafica



Conclusiones

- Los 13 mini proyectos permitieron avanzar de forma progresiva desde conceptos básicos del DOM hasta la integración de eventos, JSON, APIs externas, almacenamiento del navegador y librerías de terceros.
- El uso constante de `addEventListener()` y de funciones `async` con `fetch` demuestra que estos dos elementos son la columna vertebral del desarrollo web moderno con JavaScript.
- El proyecto integrador final (carpeta 13) refleja cómo, al combinar varias técnicas aprendidas por separado, es posible construir una pequeña aplicación funcional con sesión de usuario, consumo de APIs y visualización de datos.